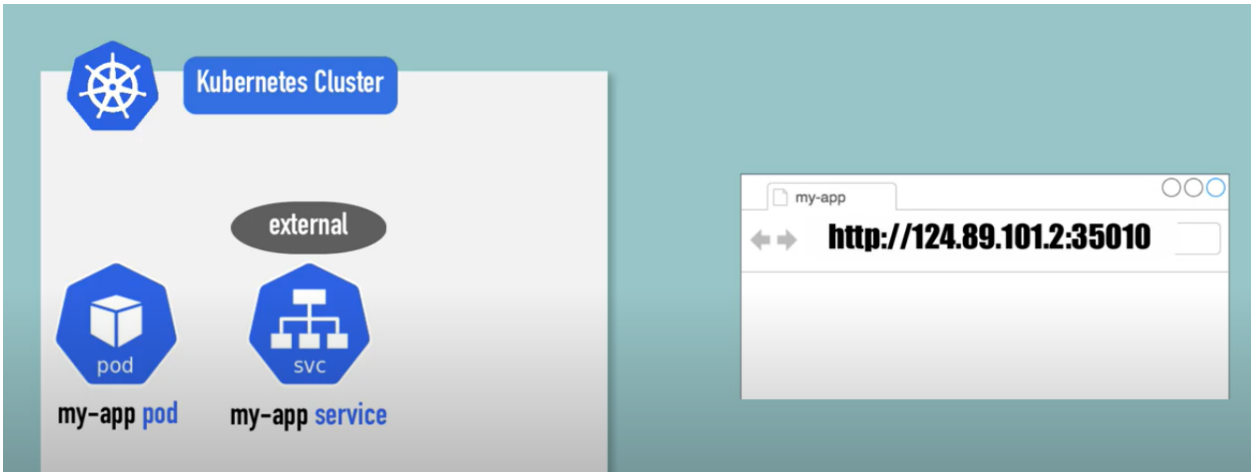
Ingress:

Path based routing

UC: We will start with a simple scenario. You are deploying an application on Kubernetes for a company that has an **online store selling products.** Your application would be available at say **my-online-store.com.**

You build the application into a Docker Image and deploy it on the kubernetes cluster as a **POD in a Deployment.** Your application needs a database so you deploy a **MySQL database as a POD** and create a service of type **ClusterIP called mysql-service to make it accessible to your application**. Your application is now working. **To make the application accessible to the outside world, you create another service, this time of type NodePort and make your application available on a high-port on the nodes in the cluster.**



In above example a port 35010 is allocated for the service. The users can now access your application using the URL:

http ://< IP of any of your nodes >:35010

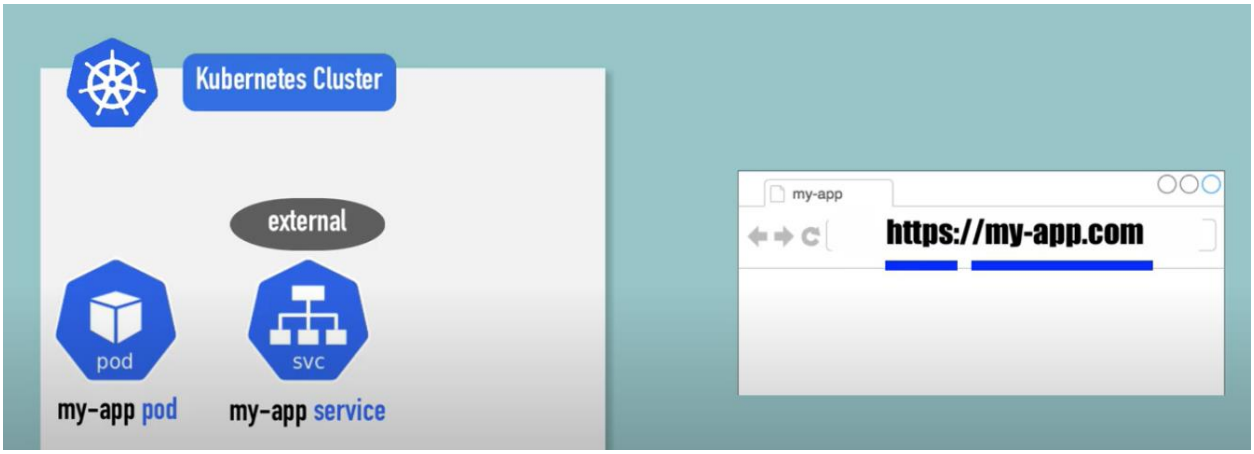
That setup works and users are able to access the application.

Whenever traffic increases, we increase the number of replicas of the POD to handle the additional traffic, and the service takes care of splitting traffic between the PODs. However, if you have deployed a production grade application before, you know that there are many more things involved in addition to simply splitting the traffic between the PODs. For example, we do not want the users to have to

type in IP address every time. So, you configure your DNS server to point to the IP of the nodes. Your users can now access your application using the URL.

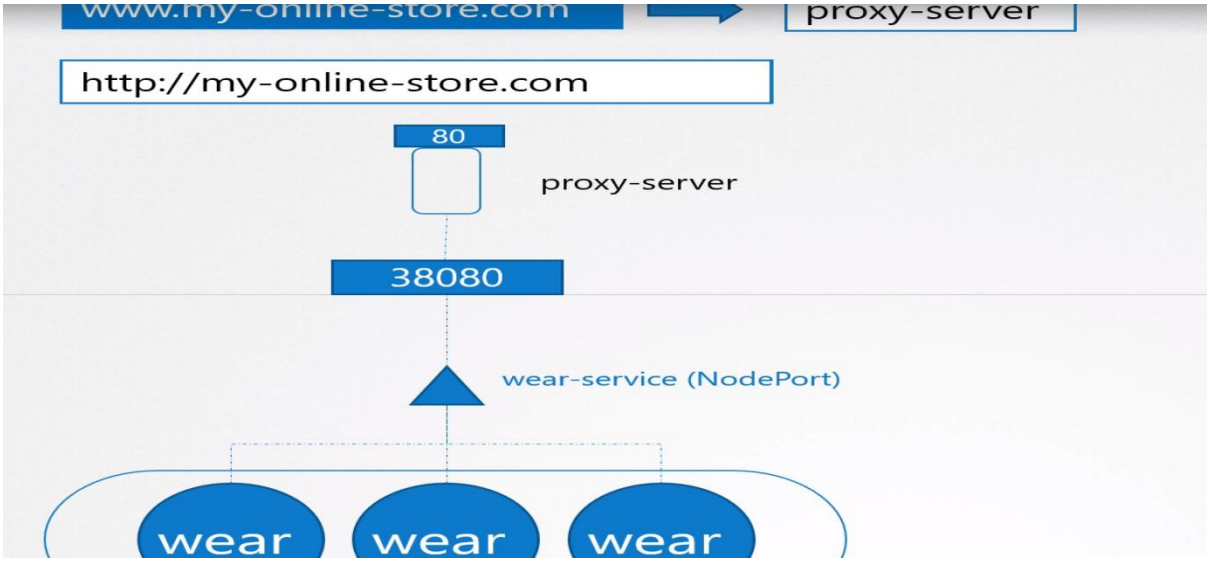
http://my-app.com:35010

Now, you don't want your users to have to remember port number either. However, Service NodePort can only allocate high numbered ports which are greater than 30,000. **So, you then bring in an additional layer between the DNS server and your cluster, like a proxy server, that proxies requests on port 80 to port 38080 on your nodes. You then point your DNS to this server, and users can now access your application by simply visiting** URL

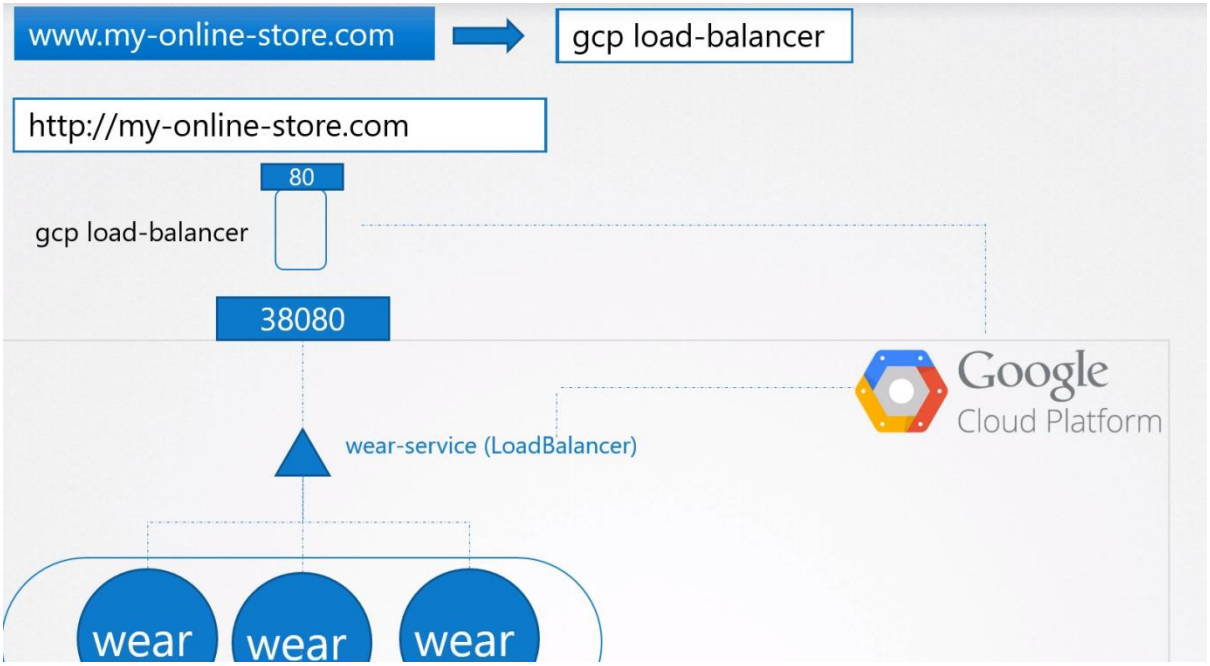


http://my-online-store.com

Now, this is if your application is hosted on-prem in your data center



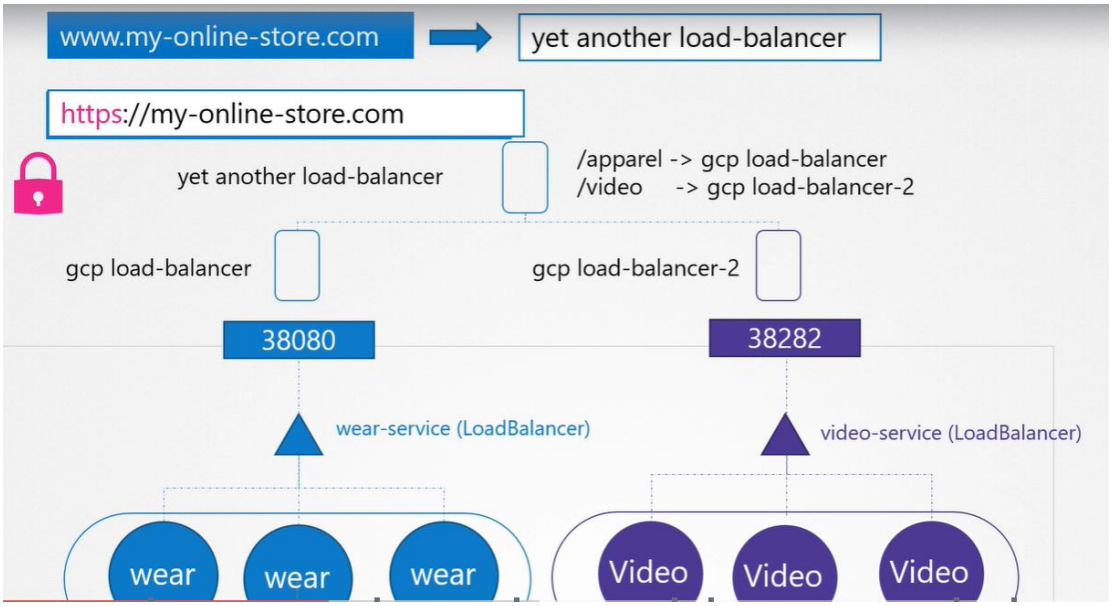
In case of cloud provider environment, instead of creating a service type of nodePort, we have to create a service type of LoadBalancer, in that case Kubernetes do the same thing did during of type nodePort additionally send a request to cloud vendor (GCP) for additional load balancer, GCP creates a LB with external IP and route the traffic to nodePort.



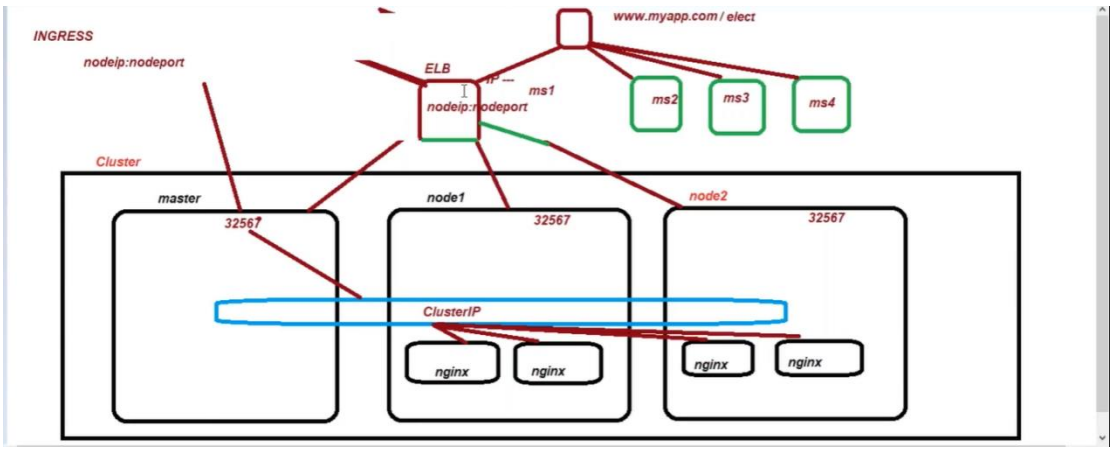
Use Case:

Use Case: Our business is growing and we have launched a new video streaming service for our customers. Your developers developed the new video streaming application as a completely different application, as it has nothing to do with the existing one. However, to share the cluster resources, you deploy the new application as a separate deployment within the same cluster. You create a service called video-service of type LoadBalancer. Kubernetes provisions port 38282 for this service and also provisions a cloud native LoadBalancer. The new load balancer has a new IP.

You need yet **another proxy** or load balancer that can re-direct traffic based on URLs to the different services. **Every time you introduce a new service you have to reconfigure the load balancer a**nd **finally you also need to enable SSL for your applications, so your users can access your application using https. Where do you configure that**? Now that's a lot of different configuration and all of these becomes difficult to manage when your application scales. It requires involving different individuals in different teams. You need to configure your firewall rules for each new service and its expensive as well, as for each service a new cloud native load balancer will be provisioned.

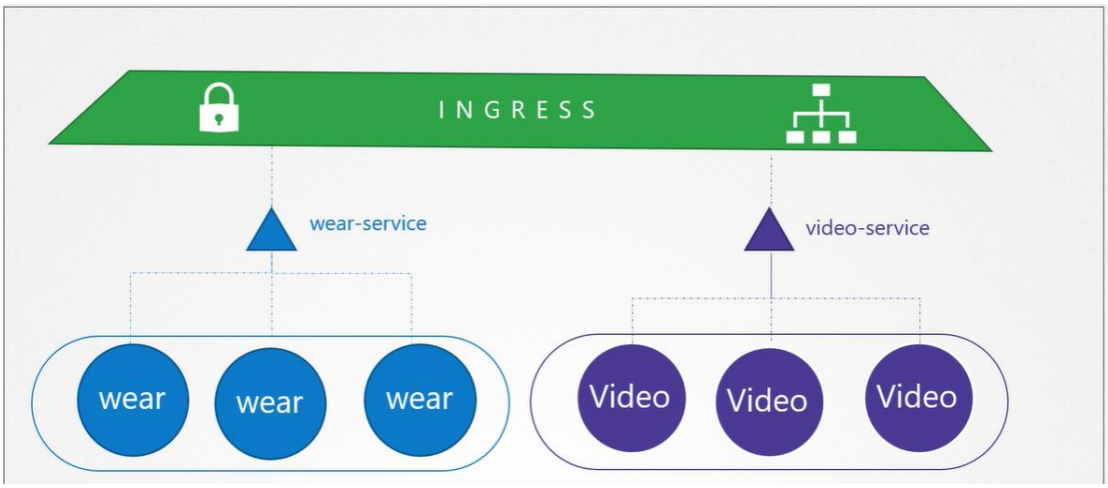


Remember you must pay for each of these load balancers and having many such load balancers can inversely affect your cloud bill is very costly solution. So how do you direct traffic between each of these load balancers based on URL the user typed in?



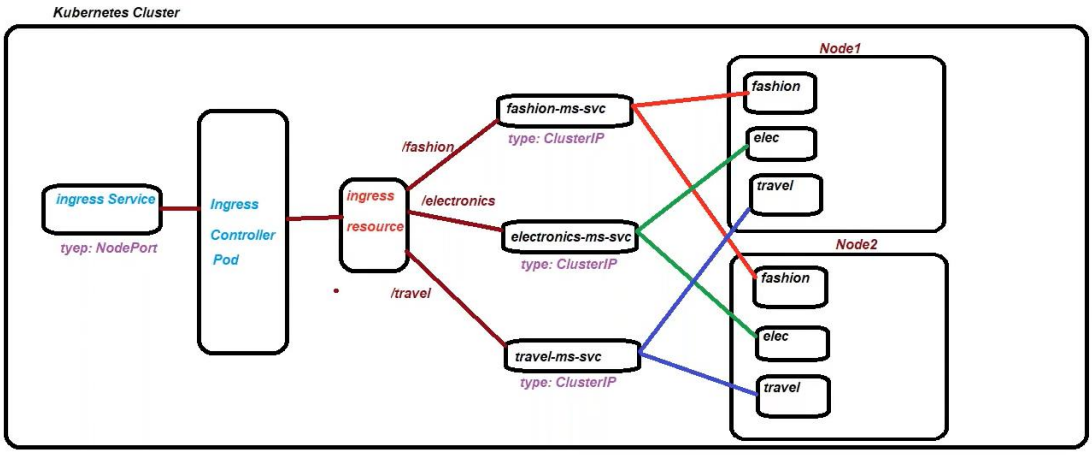
Ingress:

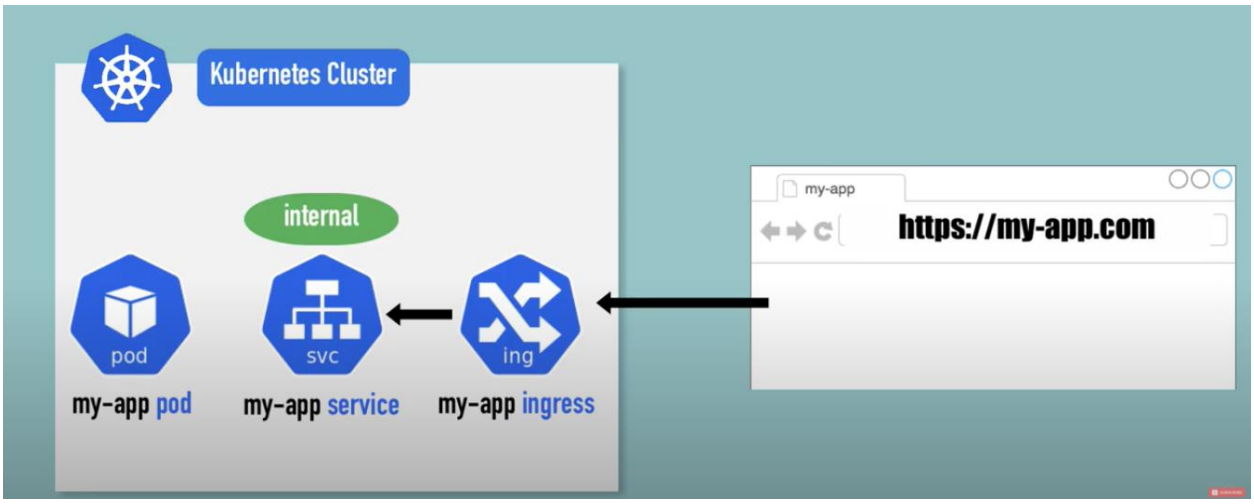
- Helps **your user access your application using a single externally accessible URL**, that you can you configure to route to different services with in your cluster based on URL paths.
- **Same time implement SSL security.**
- Think as a Layered 7 Load Balancer, built into k8s cluster that can be configured using k8s primitive just like any other objects in objects k8s.
- In cloud environment once we create an Ingress object it creates ALB.



Ingress has two parts

- Ingress Controller (Deployment- add-on):
- Ingress Resources (Pod)
- In cloud environment once we create an Ingress object it creates ALB.



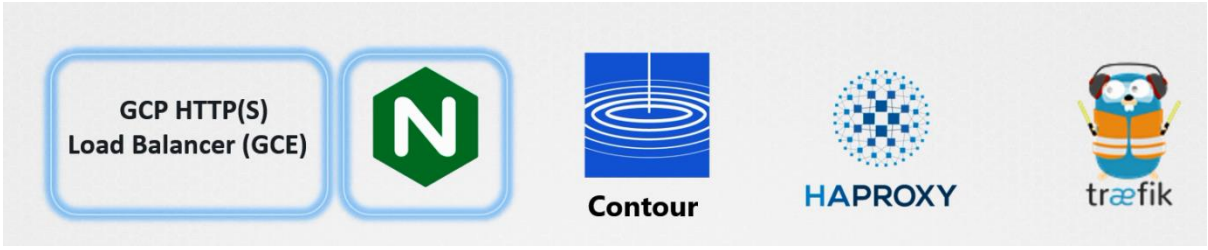


Ingress controller don't come with K8s by default, so first you have to deploy Ingress controller first.

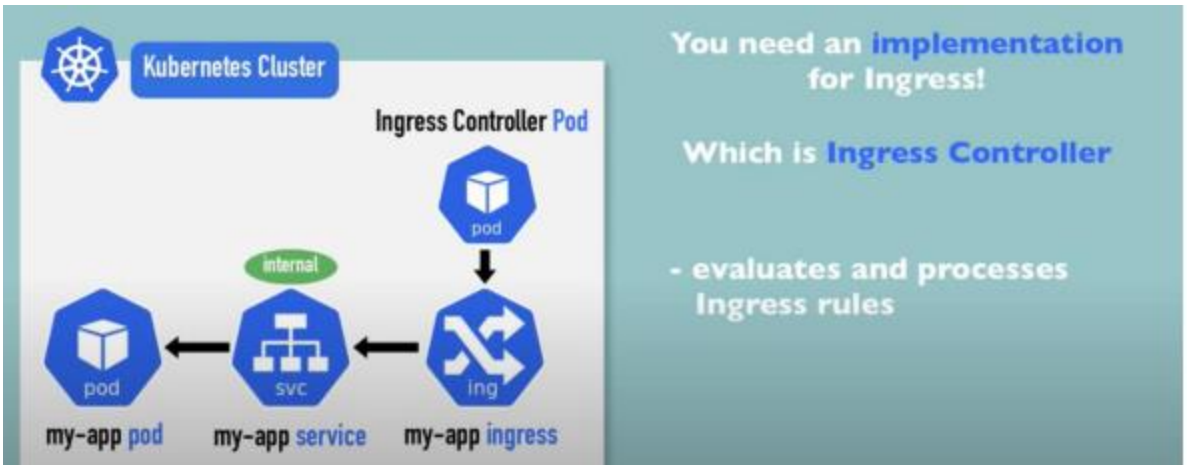
Ingress Controller: In cloud environment once we create an Ingress object it creates ALB.

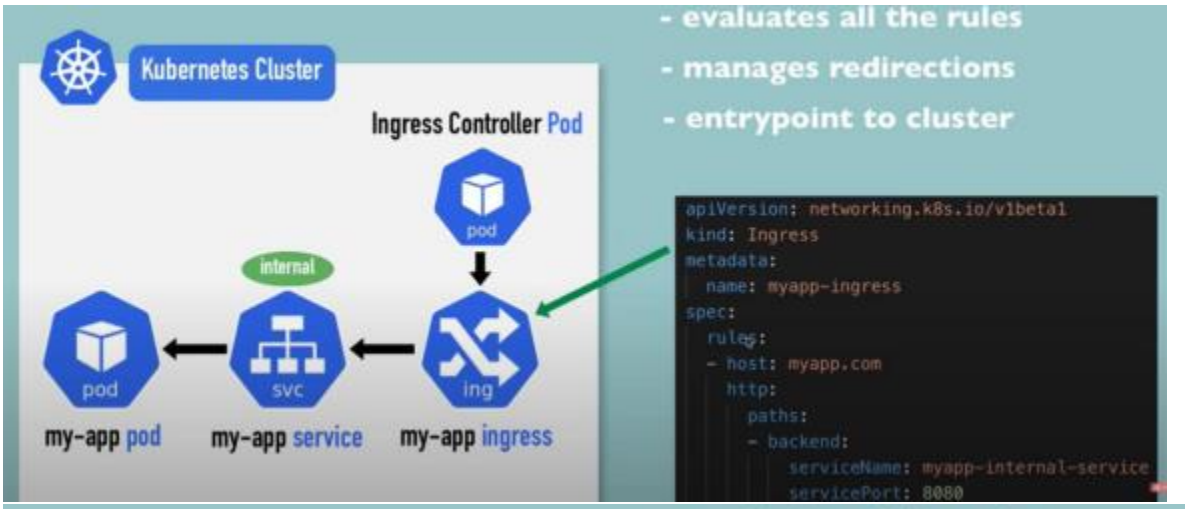
The main objective of Ingress controller is

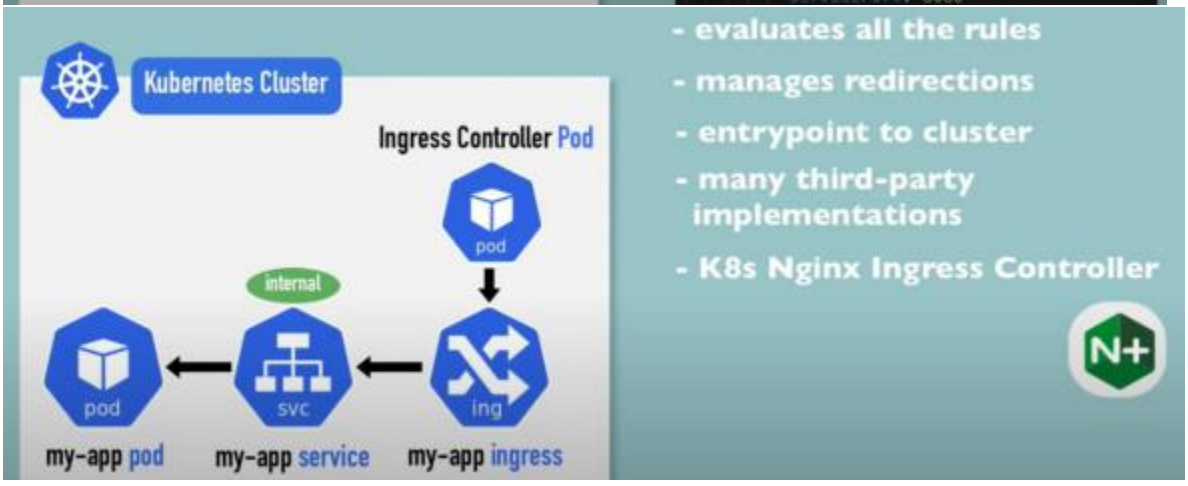
- evaluates all ingress rules.
- manage redirections
- entrypoint to cluster
- many third-party implementations
- K8s Nginx Ingress Controller

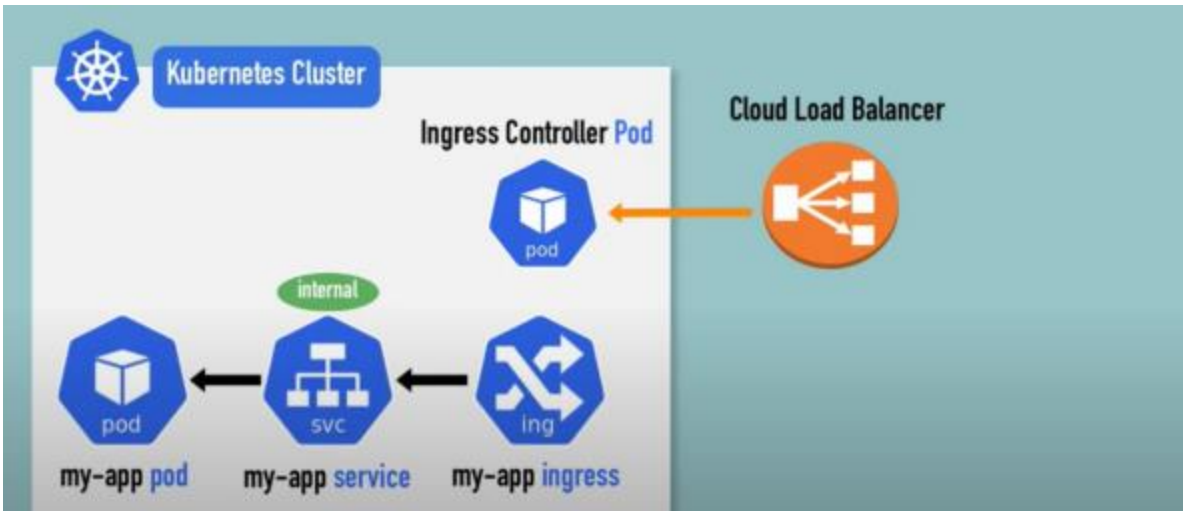


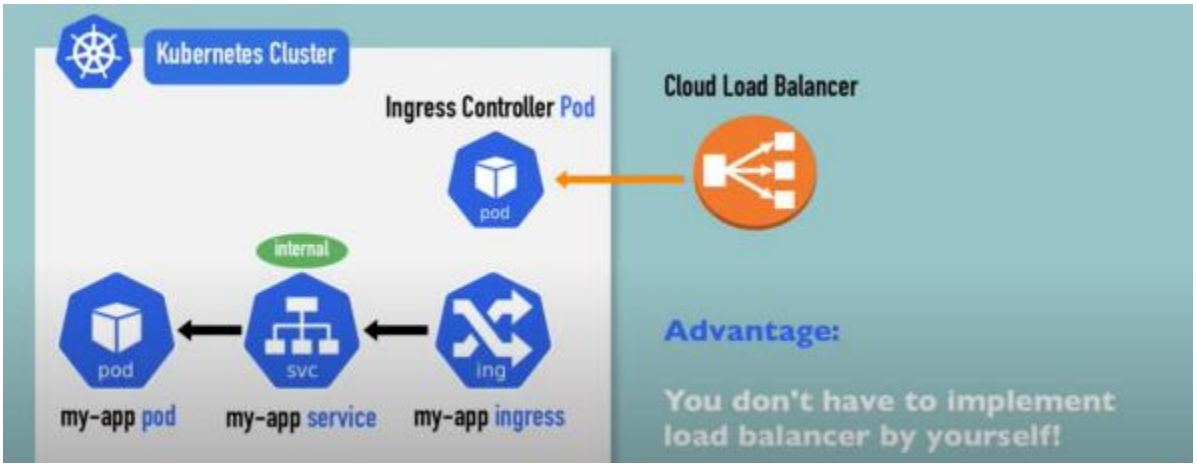
Ingress controllers are not just a LB, Ingress controller have additional intelligence build in to it to monitor K8s Cluster.

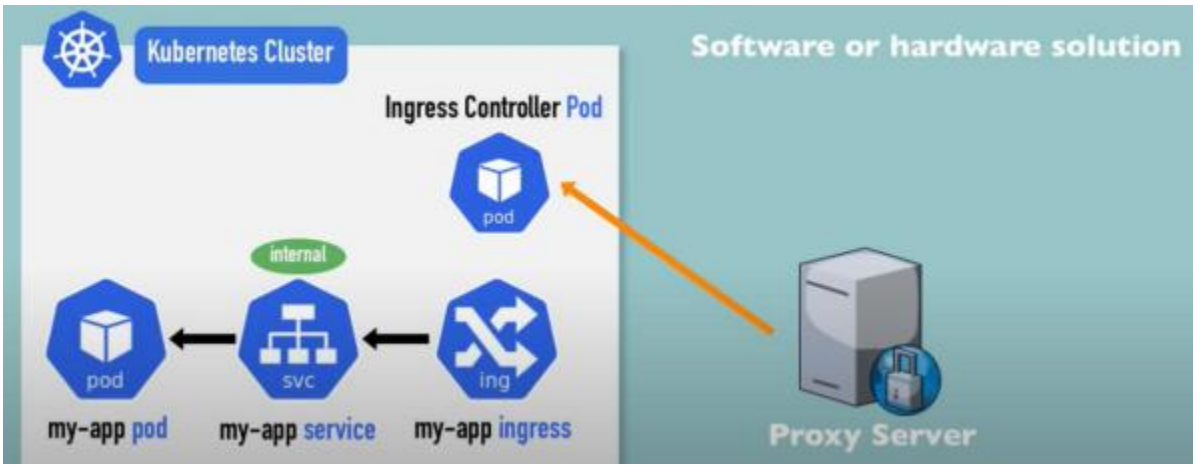

Kubernetes Cluster
Ingress Controller Pod
pod
internal
pod
svc
ing
my-app pod
my-app service
my-app ingress
You need an implementation for Ingress!
Which is Ingress Controller
- evaluates and processes Ingress rules

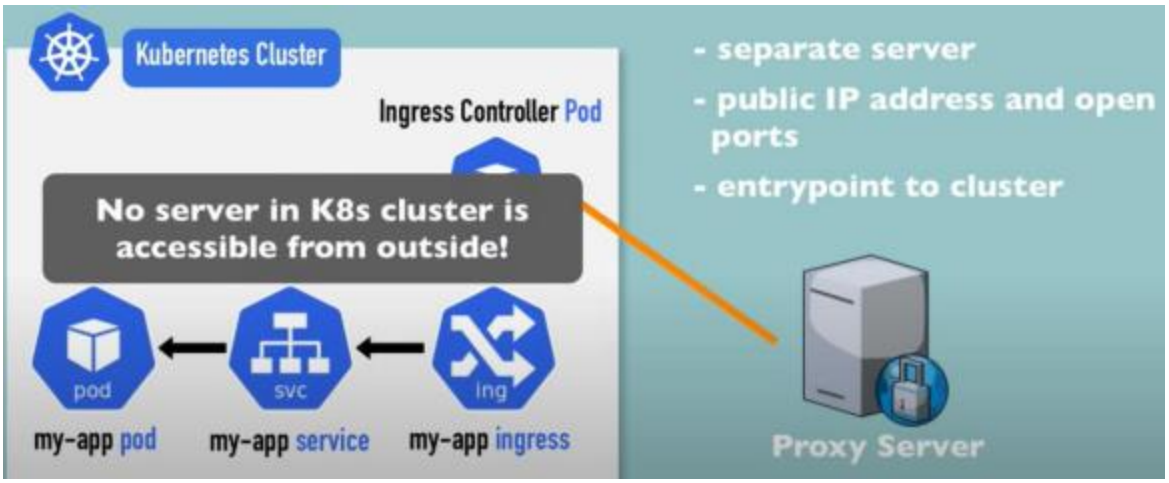

Kubernetes Cluster
Ingress Controller Pod
pod
internal
pod
svc
ing
my-app pod
my-app service
my-app ingress
- evaluates all the rules
- manages redirections
- entrypoint to cluster
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
name: myapp-ingress
spec:
rules:
- host: myapp.com
http:
paths:
- backend:
serviceName: myapp-internal-service
servicePort: 8080

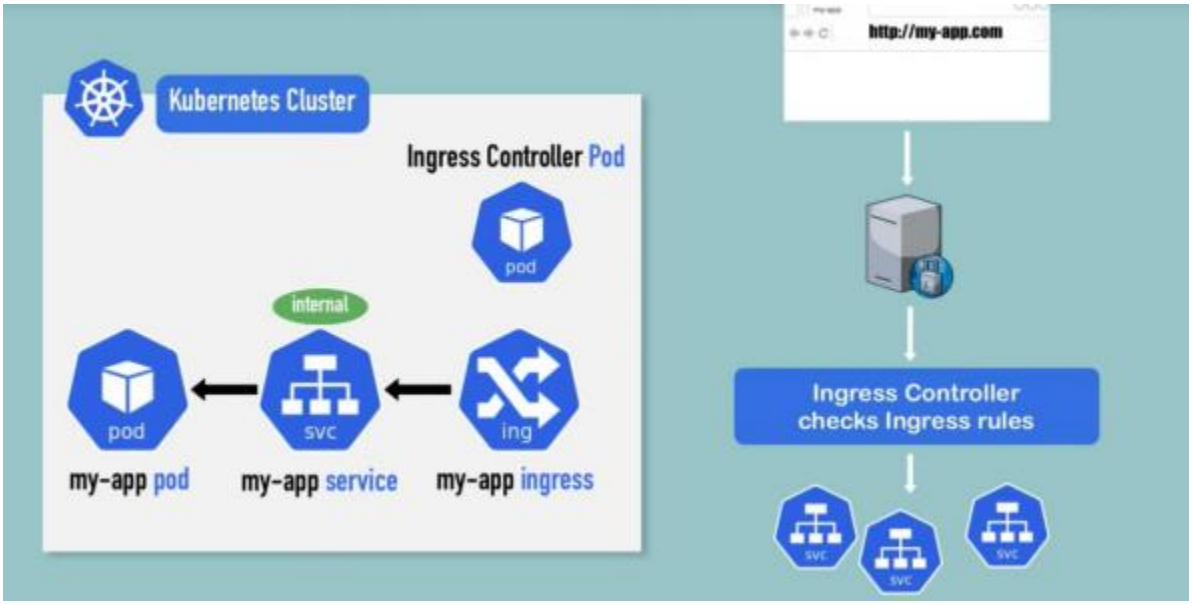

Kubernetes Cluster
Ingress Controller Pod
pod
internal
pod
svc
ing
my-app pod
my-app service
my-app ingress
- evaluates all the rules
- manages redirections
- entrypoint to cluster
- many third-party implementations
- K8s Nginx Ingress Controller
N+


Kubernetes Cluster
Ingress Controller Pod
pod
Cloud Load Balancer
internal
pod
svc
ing
my-app pod
my-app service
my-app ingress


Kubernetes Cluster
Ingress Controller Pod
pod
Cloud Load Balancer
internal
pod
svc
ing
my-app pod
my-app service
my-app ingress
Advantage:
You don't have to implement load balancer by yourself!


Kubernetes Cluster
Software or hardware solution
Ingress Controller Pod
pod
internal
pod
svc
ing
my-app pod
my-app service
my-app ingress
Proxy Server





Ingress Resources

Ingress Resources: Is a set of routing rules, forward the request to the internal service (ClusterIP).

```yaml
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
  - host: myapp.com
    http:
      paths:
      - backend:
          serviceName: myapp-internal-service
          servicePort: 8080
```

Routing rules:

Forward request to the internal service.


```yaml
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
  - host: myapp.com
    http:
      paths:
      - backend:
          serviceName: myapp-internal-service
          servicePort: 8080
```

Routing rules:

Forward request to the internal service.

Ingress:

```yaml
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
  - host: myapp.com
    http:
      paths:
      - backend:
          serviceName: myapp-internal-service
          servicePort: 8080
```

Internal Service:

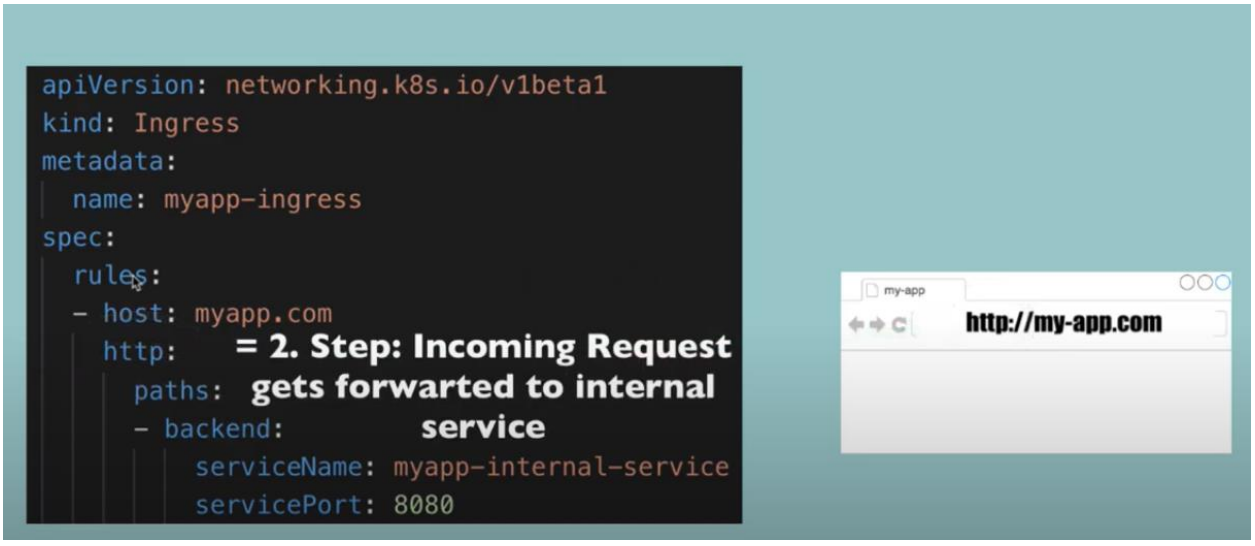
```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-internal-service
spec:
  selector:
    app: myapp
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
```

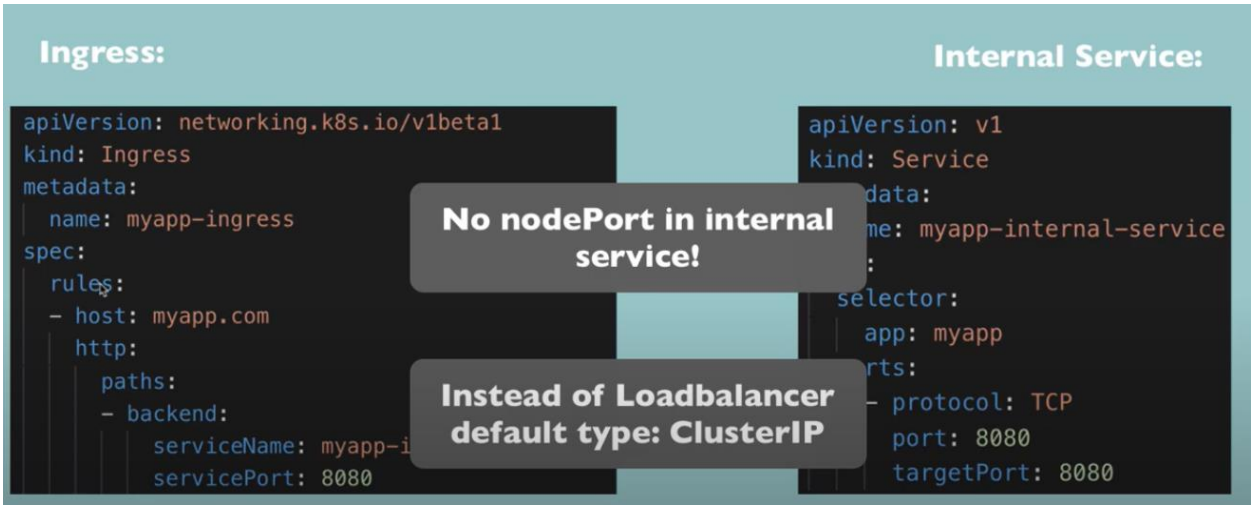
Ingress:

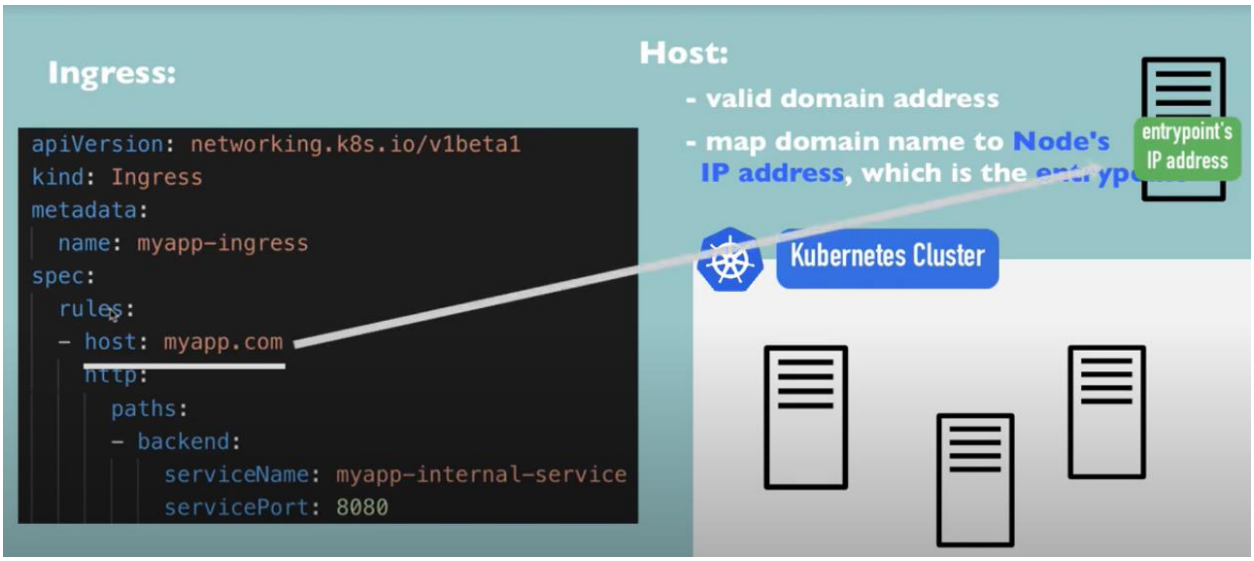
```yaml
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
  - host: myapp.com
    http:
      paths:
      - backend:
          serviceName: myapp-internal-service
          servicePort: 8080
```

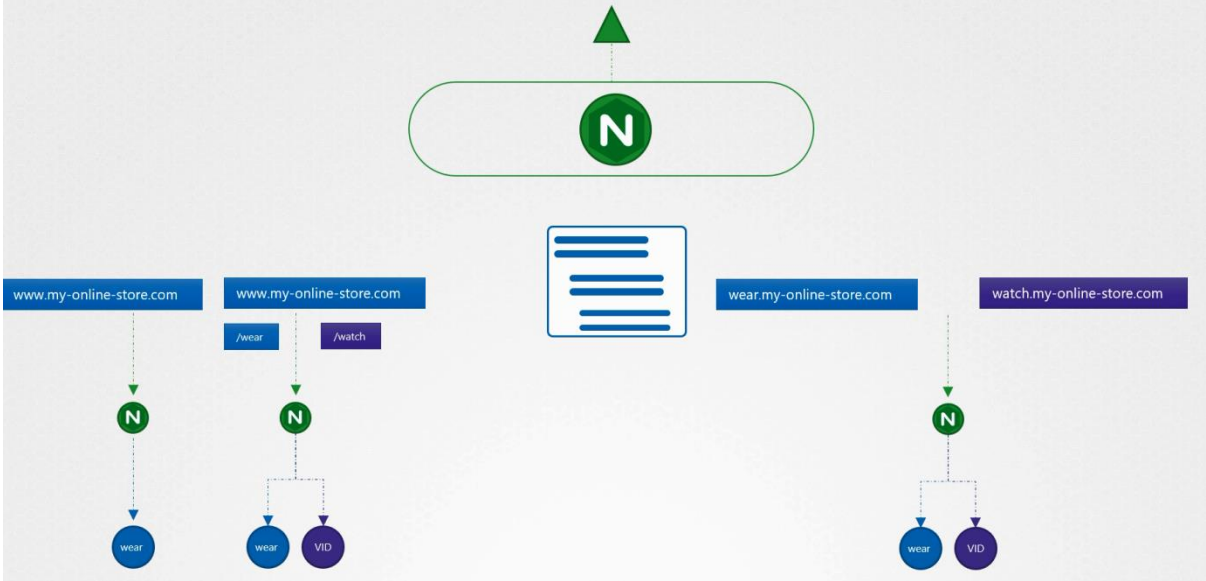
Internal Service:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-internal-service
spec:
  selector:
    app: myapp
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
```




Ingress:
Internal Service:
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
name: myapp-ingress
spec:
rules:
- host: myapp.com
http:
paths:
- backend:
serviceName: myapp-i
servicePort: 8080
No nodePort in internal service!
Instead of Loadbalancer default type: ClusterIP
apiVersion: v1
kind: Service
data:
me: myapp-internal-service
:
selector:
app: myapp
rts:
- protocol: TCP
port: 8080
targetPort: 8080


Ingress:
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
name: myapp-ingress
spec:
rules:
- host: myapp.com
http:
paths:
- backend:
serviceName: myapp-internal-service
servicePort: 8080
Host:
- valid domain address
- map domain name to Node's IP address, which is the
entrypoint's IP address
Kubernetes Cluster



UC1: If there is a single backend service, no rule (backend defines where the traffic routed)?

```
kind: Ingress
apiVersion: networking.k8s.io/v1
metadata:
  name: imgress-rule1
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - http                                  optional
        paths:
          - path: /testpath                 path means resource path in which this rule would be applied
            pathType: Prefix
            backend:
              service:
                name:
                port:
                  number:
```

UC2: Use rule when you want to route traffic based on different conditions (same URL). Each rule defines different paths

UC2.1: Single Rule 2 Paths

Our requirement is to handle all traffic coming to www.my-online-store.com

Under rules: we specify different paths- **paths is an array, one path for each URL**

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear-watch
spec:
  rules:
  - http:
      paths:
      - path: /wear
        backend:
          serviceName: wear-service
          servicePort: 80
      - path: /watch
        backend:
          serviceName: watch-service
          servicePort: 80
```

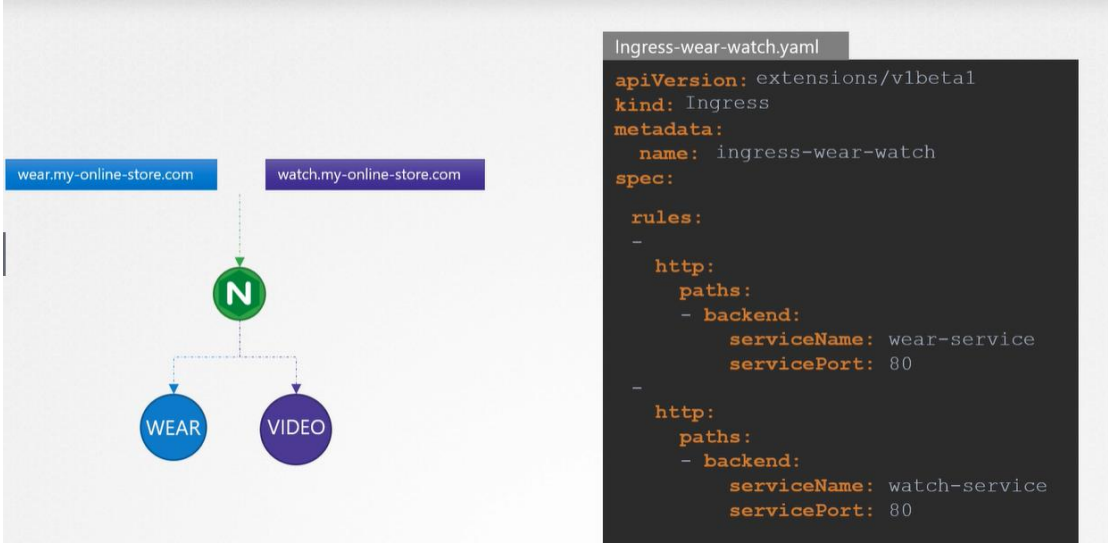
```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear-watch
spec:
  rules:
  - http:
      paths:
      - path: /wear


      - path: /watch
```

kubectl describe ingress ingress-wear-watch

```
Name:             ingress-wear-watch
Namespace:        default
Address:
Default backend:  default-http-backend:80 (<none>)
Rules:
  Host  Path  Backends
  ----  ----  --------
  *
        /wear    wear-service:80 (<none>)
        /watch   watch-service:80 (<none>)
Annotations:
Events:
  Type    Reason  Age   From                      Message
  ----    ------  ----  ----                      -------
  Normal  CREATE  14s   nginx-ingress-controller  Ingress default/ingress-wear-watch
```

UC3: Using the domain names (hostPath)

2 Rules (for each domain), 1 Path each



Ingress-wear-watch.yaml

```yaml
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear-watch
spec:
 rules:
 -
   http:
     paths:
     - backend:
         serviceName: wear-service
         servicePort: 80
 -
   http:
     paths:
     - backend:
         serviceName: watch-service
         servicePort: 80
```

If you don’t specify the host, it considered as * means for all incoming traffic.

Ingress-wear-watch.yaml

```yaml
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear-watch
spec:
  rules:
  - http:
      paths:
      - path: /wear
        backend:
          serviceName: wear-service
          servicePort: 80
      - path: /watch
        backend:
          serviceName: watch-service
          servicePort: 80
```

```yaml
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ingress-wear-watch
spec:
 rules:
 - host: wear.my-online-store.com
   http:
     paths:
     - backend:
         serviceName: wear-service
         servicePort: 80
 - host: watch.my-online-store.com
   http:
     paths:
     - backend:
         serviceName: watch-service
         servicePort: 80
```

UC4: Secure Ingress

You can secure an Ingress by specifying a Secret that contains a TLS private key and certificate. The Ingress resource only supports a single TLS port, 443, and assumes TLS termination at the ingress point (traffic to the Service and its Pods is in plaintext). If the TLS configuration section in an Ingress specifies different hosts, they are multiplexed on the same port according to the hostname specified

through the SNI TLS extension (provided the Ingress controller supports SNI). The TLS secret must contain keys named tls.crt and tls.key that contain the certificate and private key to use for TLS.

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: testsecret-tls
  namespace: default
data:
  tls.crt: base64 encoded cert
  tls.key: base64 encoded key
type: kubernetes.io/tls
```

Referencing this secret in an Ingress tells the Ingress controller to secure the channel from the client to the load balancer using TLS.

You need to make sure the TLS secret you created came from a certificate that contains a Common Name (CN), also known as a Fully Qualified Domain Name (FQDN) for https-example.foo.com.

Note: Keep in mind that TLS will not work on the default rule because the certificates would have to be issued for all the possible sub-domains.

Therefore, hosts in the tls section need to explicitly match the host in the rules section.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-example-ingress
spec:
  tls:
  - hosts:
      - https-example.foo.com
    secretName: testsecret-tls
  rules:
  - host: https-example.foo.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: service1
            port:
              number: 80
```

What is Proxy Server?

Proxy Server acts as a gateway between users and the Internet. This is an intermediary server between the end user and the website they visit. Proxy servers offer different functions, security and privacy depending on your needs or company policy.

If you are using a proxy server, Internet traffic will pass through the proxy server along its route to the address you require. After that, this request will return to the same proxy server (also the exception to this rule) and that proxy server forwards the data received from the website to the user.

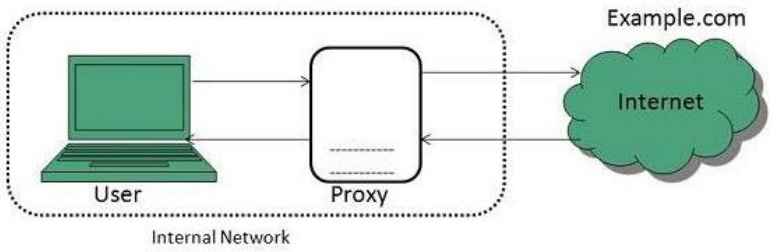
Modern proxy servers do more than just forward web requests, they also perform data security and increase network performance. Proxy servers act as firewalls and web filters, providing shared network connectivity and cache data to speed up common requirements. A good proxy server will protect users and intranets from unwanted internet. Finally, proxy servers can provide high levels of privacy.

Type of Proxies

Following table briefly describes the type of proxies:

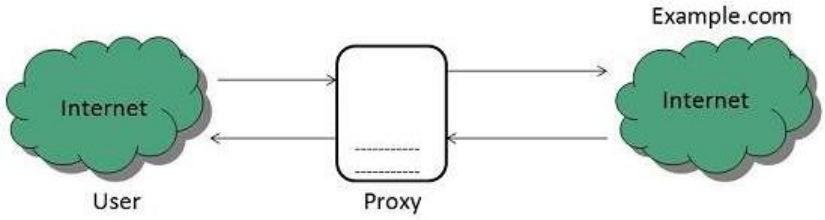
Forward Proxies

In this the client requests its internal network server to forward to the internet.



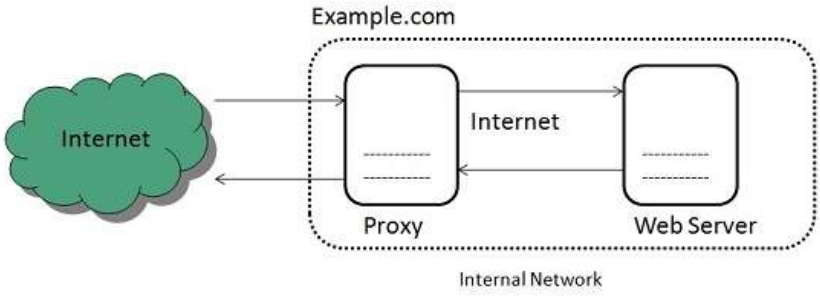
Open Proxies

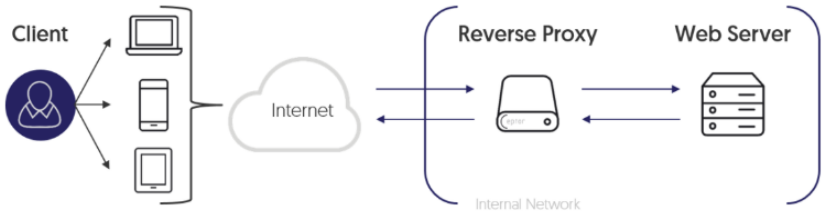
Open Proxies helps the clients to conceal their IP address while browsing the web.



Reverse Proxies

In this the requests are forwarded to one or more proxy servers and the response from the proxy server is retrieved as if it came directly from the original Server.





How Ingress Controller Map with Ingress Rule?

By default, we allow only one ingress controller in cluster, can edit ingress controller name in ingress rule.

How app1 talk to app2? // Internal communication

If app1 wants to talk to app2, then you have to refer through app1 svc.

Ingress don't do the autoscaling just forward the request.